

Report

- Assignment 1 -

Artificial Intelligence Dept.

2277025 Eunsang Lee

Problem Definition

Classifying images as smoking or not-smoking

Development Environment

OS : Goggle Colab

Python : 3.11.12 (default version in colab)

Libraries

- kagglehub : 0.3.12
- tensorflow : 2.18.0
- sklearn : 1.6.1
- matplotlib : 3.10.0

for a detailed list of required packages and their versions, please refer to the 'requirements.txt' file provided.

Dataset Description

Dataset is from Kaggle.

Here is the Link : <https://www.kaggle.com/datasets/sujaykapadnis/smoking>

The folders are divided into 'Training', 'Validation', and 'Testing'. And each dataset is included. In addition, the images are divided equally into two classes (Smoking and NotSmoking).

- Training : 716 images (358 Smokings and 358 NotSmokings)
- Validation : 180 images (90 Smokings and 90 NotSmokings)
- Testing : 224 images (112 Smokings and 112 NotSmokings)

I used the 'kagglehub' library to retrieve the dataset directly in Colab. Then, I processed

the data using the 'tensorflow.keras.preprocessing.image.ImageDataGenerator' module to make it suitable for the model. To address the issue of a small training dataset, I applied data augmentation techniques to the training dataset.

Model Description

Due to the limited time for implementing a complex model, I created a simple CNN model. Additionally, I thought the convolutional filters can effectively capture the features of the cigarette.

It has three convolution layers for feature extraction and two dense layers for classification. To enhance training stability and prevent overfitting, I also put batch normalization, max pooling, and dropout layers.

Here is the detail about the model.

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 148, 148, 16)	448
batch_normalization (BatchNormalization)	(None, 148, 148, 16)	64
max_pooling2d (MaxPooling2D)	(None, 74, 74, 16)	0
conv2d_2 (Conv2D)	(None, 72, 72, 32)	4,640
batch_normalization_1 (BatchNormalization)	(None, 72, 72, 32)	128
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 32)	0
conv2d_3 (Conv2D)	(None, 34, 34, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 34, 34, 64)	256
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 64)	0
flatten (Flatten)	(None, 18496)	0
dense (Dense)	(None, 512)	9,470,464
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 1)	513
Total params: 9,495,009 (36.22 MB)		
Trainable params: 9,494,785 (36.22 MB)		
Non-trainable params: 224 (896.00 B)		

Each convolutional layer extracts increasingly complex features as the network deepens, and all activation functions are unified as ReLU. The He initialization is used to efficiently set the initial weights, contributing to more stable training.

In the dense layers, a dropout rate of 50% is applied to prevent overfitting, and the final

output layer uses a sigmoid function to produce a probability value for binary classification.

Evaluation Methods & Metrics

During training, the model is trained on the training dataset and evaluated on the validation set. An EarlyStopping callback is used to prevent overfitting. If the loss does not decrease by at least 0.001 for 10 consecutive epochs, training is stopped early. The final weights are restored from the epoch with the best performance based on validation loss.

Loss is calculated by the binary cross-entropy, because the problem has just two classes. This loss function measures the difference between the predicted probability and the actual label, and is commonly used for problems where the output is either 0 or 1.

During training, accuracy was used as the evaluation metric to quickly monitor the model's performance at each epoch. And for the final test dataset, I used accuracy, precision, recall, and F1-score to evaluate the model from multiple perspectives.

Results & Interpretation

The model exhibited poor performance on the test set with the following metrics

- Accuracy : 0.5223
- Precision : 0.5210
- Recall : 0.5536
- F1-score : 0.5368

Following graph is the loss during training.



The model is converging to a loss of around 0.4. I think it is because the model is too

simple to capture the underlying patterns in the data, or due to the small size of the dataset, even using the data augmentation. I think it can be resolved by using a more complex model or increasing the training data.

Link for the remote source repository

<https://github.com/twoSilverUp/MLOps/tree/Assignment1>